Fault-Tolerant Distributed Systems

Lab 1 – From one server to naive replication

2026/2027

Deliverable: lab record, due by the end of Friday, 30 October (see the last section).

Goal Replication is the obvious answer to server failures. In this lab you will state what a key-value service should do, build it with one server, break it, and then try to make it fault tolerant by running several copies. Different groups will try different replication strategies and compare the results at the end of the class.

Learning outcomes State the specification and assumptions of a replicated service. Use fault injection and execution histories as evidence. Distinguish correctness from availability. Recognize why naive replication strategies fail and what they trade off.

How we work Each part starts with *thinking without a computer*: individually, then as a group. Only then do you implement and run. You may use AI assistants for implementation and analysis, following the workflow in Part 2. Keep a single shared document per group with your answers; you will need it in the following labs.

Setup Install Maelstrom 0.2.4 following its instructions[1]: install Java (17 or later), `gnuplot`, `graphviz` and `git`, and extract `maelstrom.tar.bz2` from the release page.[2] You also need Python (3.8 or later). Download `kit.tar.bz2` from eLearning and extract it into the same directory as Maelstrom, so that `kit` and `maelstrom` are side by side. Read `kit/README.md`, then check the installation by running, inside `kit`:

```
./run.sh echo echo.py
```

The output must end with *Everything looks good!*

Part 0 – Predict *(10 min)*

Individually, without discussing and without AI, write down short answers:

1. A client sends `write(x, 5)` to a single server, which crashes at some moment. List the possible outcomes *from the client's point of view*. Can the client always tell which one happened?
2. Complete the sentence: "The key-value store works correctly if . . . "
3. We now run three copies of the server. Which problems of question 1 go away? Which new problems appear?

Keep these answers; you will revisit them at the end.

Part 1 – Specify the service *(20 min)*

We use Maelstrom's `lin-kv` workload: clients send `read`, `write` and `cas` (compare-and-set) requests on integer keys (see `kit/AGENTS.md` for the messages).

As a group, compare your answers to Part 0 and write:

1. **Specification.** What each operation must return, first for a single client, then with several clients. In particular: what may a `read` return when it is concurrent with a `write` of the same key? What must it return if it starts *after* a write has been acknowledged?

[1] `https://github.com/jepsen-io/maelstrom/blob/main/doc/01-getting-ready/index.md`
[2] `https://github.com/jepsen-io/maelstrom/releases/tag/v0.2.4`

2. **Assumptions.** Number of servers; what can fail and how (crash? message loss? delay?); what you assume about timing.
3. **A first claim**, using this template:

   > *Under assumptions . . . , if at most f . . . fail by . . . , the service guarantees . . . . Evidence: . . . . Not covered: . . .*

   You have no evidence yet: leave it empty for now.

Checkpoint: the instructor will ask some groups to read their specification to the class.

Part 2 – One server *(30 min)*

Working with an AI assistant

1. **Decide first.** The design comes from your group, not from the assistant.
2. **Give context.** Give the assistant `kit/AGENTS.md` (many tools read it automatically) and your design. Ask it to restate the design and its assumptions *before* writing code, and correct it where it differs from yours.
3. **Review the change.** Every member must be able to explain every line; any member may be asked to.
4. **The run is the evidence.** "This should work" is not evidence: run it and read the results.
5. **Hypothesis first.** When a run fails, write your own explanation before asking the assistant, then compare.
6. **Log what matters.** Note the prompts that mattered and what you had to correct or reject.

1. Implement a single-server `lin-kv` node (`kv.py`) using `ms.py`. Test it with:

   ```
   ./run.sh lin-kv kv.py --node-count 1
   ```

Who is who in a Maelstrom run

- **Nodes** `n0`, `n1`, ... are your servers.
- **Processes** `0`, `1`, ... are the logical clients. Each performs one operation at a time and always talks to the same node. By default, twice as many processes as nodes are active at any moment (`--concurrency`). `history.txt`, `timeline.html` and `linear.svg` identify operations by process; `./timeline.py --history` shows them as `p0`, `p1`, ...
- When an operation ends as *info* (its outcome is unknown), its process cannot continue: the checker treats each process as doing one operation after the other, and cannot tell whether the last one took effect. The process is retired and replaced by a new one with a higher number (e.g. `p3` by `p9`), which talks to the same node.
- **Client ids** `c6`, `c7`, ... are the network names of the clients, found in the messages your nodes receive (`msg.src`) and in `messages.svg`. They do not match process numbers: `./timeline.py --history` shows the node each operation was sent to, followed by the events it caused there.

2. *Predict, then run.* The server crashes 5 seconds into the test:

   ```
   MS_CRASH="n0@5" ./run.sh lin-kv kv.py --node-count 1
   ```

   Will the checker say *valid*? Then look at `:ok-count`, `:fail-count` and `:info-count` in `store/latest/results.edn` and open `store/latest/timeline.html`. A *fail* is a definite error reply, which includes normal answers such as "key does not exist" or a `cas` mismatch; an *info* is an operation whose outcome the client does not know. Why can the client not know, and how does this relate to Part 0, question 1?
3. Is your single-server store *correct* after the crash? Is it *available*? Which part of your claim from Part 1 does this run support, and which part does it contradict?

4. *Predict, then run.* The same program with three nodes and no coordination between them:

   ```
   ./run.sh lin-kv kv.py
   ```

   Open `store/latest/independent/*/linear.svg` and explain the anomaly with your own words.

Part 3 – Design a replication strategy *(20 min)*

Each group is assigned one of the strategies below. All servers keep a full copy of the store and any server may receive client requests.

A *Update, reply, propagate.* The receiving server executes the request, replies to the client, and sends the update to all other servers.

B *Update, propagate, wait for all.* The receiving server executes the request and sends the update to all others; it replies to the client only when all have acknowledged it.

C *Primary for writes.* `write` and `cas` are forwarded to a fixed primary (`n0`), which executes them and sends the update to the others. Any server answers `read` from its own copy.

D *Primary for everything.* Every request is forwarded to `n0`, which executes it and sends the update to the others.

As a group, without a computer:

1. Draw a space-time diagram of one `write` followed by a `read` from another client at another server.
2. Decide the details your strategy leaves open (e.g. what is sent, what a server does when an update arrives, how `cas` is handled).
3. Predict, for each scenario in the table below, whether the checker will report *valid*, and whether clients will be able to complete operations. Justify each prediction in one sentence.

Scenario	Command options
No faults	(none)
Latency, more load	`--latency 50 --rate 50`
Primary/any server crashes	`MS_CRASH="n0@5"`
Another server crashes	`MS_CRASH="n1@5"`
Network partitions	`--nemesis partition`

Part 4 – Build and test *(40 min)*

1. Implement your strategy, following the AI workflow from Part 2.
2. Run the five scenarios of Part 3 and record, for each, the verdict and the operation counts next to your prediction.
3. Anomalies depend on timing and load: a valid run does not show that a strategy is correct. Repeat scenarios; for example, the following runs the same test three times:

   ```
   ./run.sh lin-kv a.py --latency 50 --rate 50 --test-count 3
   ```

   If all runs are valid, try increasing `--rate` and `--latency`. When a prediction was wrong, write down why before moving on.
4. Add `trace()` calls at the points that matter for your strategy (e.g. when an update is sent and applied). To see the trace events of the last run in time order, together with the client operations (`p3` is client process 3) and the partitions, use:

   ```
   ./timeline.py --history
   ```

   Add `--key 0` or `--process 3` to focus on one key or one client.

Part 5 – Explain one execution *(25 min)*

Choose the most interesting run you obtained, preferably one reported as invalid. If all your runs were valid, explain why you believe that is not a coincidence, and which scenario could break your strategy.

1. Using `linear.svg`, `messages.svg`, `timeline.html` and `./timeline.py --history`, reconstruct the execution: which clients, servers and messages were involved, in which order. `linear.svg` and `timeline.html` identify clients by process number (`p3` in `./timeline.py`); `messages.svg` and your trace events use client ids (`c6`). An `invoke` is normally followed by the `request` that it caused at that node.
2. Draw it as a space-time diagram and mark the point where things went wrong.
3. Does this execution violate *your* specification from Part 1? If the checker rejects an execution that your specification allows, either your specification or the checker is using a different notion of correctness: which one, and what is it?
4. Revise the claim from Part 1, now with evidence and with what is not covered.

Part 6 – Compare across groups *(20 min)*

Each group adds its results to the class table (strategy, scenario, predicted, observed). Then discuss:

1. Is any strategy correct in every scenario? Is any strategy available in every scenario? Is any both?
2. Strategy B waits for every server. What does that buy, and what does it cost when a server crashes?
3. Strategies C and D use a primary. What happens when the primary crashes, and how could another server take over safely?
4. Is there a middle ground between waiting for one server and waiting for all of them?

Exit record *(5 min)*

Each group writes down, in four lines: (1) the claim considered; (2) the strongest evidence obtained; (3) one limitation or unresolved question; (4) the next action.

Deliverable: lab record

By the end of Friday, 30 October, each group submits on eLearning a lab record of at most two pages, using the template `record.tex` in the kit, together with the code and the `store/` directories of the runs it cites (only those). Finish at home what you did not complete in class. For this lab:

1. **Claim**: your revised claim from Part 5.
2. **Design and predictions**: your strategy, and the table of Part 3 with predicted and observed results.
3. **Evidence**: the execution you explained in Part 5, as an annotated space-time diagram.
4. **Limitations and open questions**: what your runs do not show, and what you would test next.
5. **AI log**: the prompts that mattered and what you had to correct or reject.
6. **Contributions**: one line per member.

Lab records are kept as a portfolio and are the basis of the final discussion of the practical work, in which each member is asked individually to explain (or extend) what the group did.

Optional Before running your strategy under partitions, ask an AI assistant whether it will be reported as linearizable and why; compare its answer with the result, and add it to your AI log.